Chapter 2: Writing Test Functions

Assertions, Test Discovery, Exceptions, and Parametrization

Source: Python Testing with pytest (Brian Okken) | Study Edition for Autonomous Calendar Scheduler (Week 1)

Table of Contents & Core Learning Outline

• **Testing a Python Package** — Structuring project source code and tests directory

• **Using assert Effectively** — Natural Python assertions with detailed failure introspections

• **Expecting Exceptions** — Testing failure paths with pytest.raises() and match expressions

• **Structuring Test Files** — File naming conventions and automatic test discovery

• **Test Markers & Filtering** — Grouping tests with @pytest.mark and running subsets (-m, -k)

• **Parametrized Testing** — @pytest.mark.parametrize for testing multiple inputs against single test logic

---

Print & Study Note: *This document contains the full unabridged chapter text following this Table of Contents page. Focus on understanding the core design patterns and self-implementing the exercises without copy-pasting AI code.*

Chapter 2
Writing Test Functions

In the last chapter, you got pytest up and running. You saw how to run it against files and directories and how many of the options worked. In this chapter, you'll learn how to write test functions in the context of testing a Python package. If you're using pytest to test something other than a Python package, most of this chapter still applies.

We're going to write tests for the Tasks package. Before we do that, I'll talk about the structure of a distributable Python package and the tests for it, and how to get the tests able to see the package under test. Then I'll show you how to use assert in tests, how tests handle unexpected exceptions, and testing for expected exceptions.

Eventually, we'll have a lot of tests. Therefore, you'll learn how to organize tests into classes, modules, and directories. I'll then show you how to use markers to mark which tests you want to run and discuss how builtin markers can help you skip tests and mark tests as expecting to fail. Finally, I'll cover parametrizing tests, which allows tests to get called with different data.

Testing a Package

We'll use the sample project, Tasks, as discussed in *The Tasks Project*, to see how to write test functions for a Python package. Tasks is a Python package that includes a command-line tool of the same name, tasks.

Appendix 4, *Packaging and Distributing Python Projects* includes an explanation of how to distribute your projects locally within a small team or globally through PyPI, so I won't go into detail of how to do that here; however, let's take a quick look at what's in the Tasks project and how the different files fit into the story of testing this project.

Following is the file structure for the Tasks project:

```
tasks_proj/
├── CHANGELOG.rst
├── LICENSE
├── MANIFEST.in
├── README.rst
├── setup.py
├── src
│   └── tasks
│       ├── __init__.py
│       ├── api.py
│       ├── cli.py
│       ├── config.py
│       ├── tasksdb_pymongo.py
│       └── tasksdb_tinydb.py
└── tests
    ├── conftest.py
    ├── pytest.ini
    ├── func
    │   ├── __init__.py
    │   ├── test_add.py
    │   └── ...
    └── unit
        ├── __init__.py
        ├── test_task.py
        └── ...
```

I included the complete listing of the project (with the exception of the full list of test files) to point out how the tests fit in with the rest of the project, and to point out a few files that are of key importance to testing, namely conftest.py, pytest.ini, the various __init__.py files, and setup.py.

All of the tests are kept in tests and separate from the package source files in src. This isn't a requirement of pytest, but it's a best practice.

All of the top-level files, CHANGELOG.rst, LICENSE, README.rst, MANIFEST.in, and setup.py, are discussed in more detail in Appendix 4, *Packaging and Distributing Python Projects*. Although setup.py is important for building a distribution out of a package, it's also crucial for being able to install a package locally so that the package is available for import.

Functional and unit tests are separated into their own directories. This is an arbitrary decision and not required. However, organizing test files into multiple directories allows you to easily run a subset of tests. I like to keep functional and unit tests separate because functional tests should only break if we are intentionally changing functionality of the system, whereas unit tests could break during a refactoring or an implementation change.

The project contains two types of __init__.py files: those found under the src/ directory and those found under tests/. The src/tasks/__init__.py file tells Python that the directory is a package. It also acts as the main interface to the package when someone uses import tasks. It contains code to import specific functions from api.py so that cli.py and our test files can access package functionality like tasks.add() instead of having to do tasks.api.add().

The tests/func/__init__.py and tests/unit/__init__.py files are empty. They tell pytest to go up one directory to look for the root of the test directory and to look for the pytest.ini file.

The pytest.ini file is optional. It contains project-wide pytest configuration. There should be at most only one of these in your project. It can contain directives that change the behavior of pytest, such as setting up a list of options that will always be used. You'll learn all about pytest.ini in Chapter 6, *Configuration*.

The conftest.py file is also optional. It is considered by pytest as a "local plugin" and can contain hook functions and fixtures. Hook functions are a way to insert code into part of the pytest execution process to alter how pytest works. Fixtures are setup and teardown functions that run before and after test functions, and can be used to represent resources and data used by the tests. (Fixtures are discussed in Chapter 3, *pytest Fixtures* and Chapter 4, *Builtin Fixtures*, and hook functions are discussed in Chapter 5, *Plugins*.) Hook functions and fixtures that are used by tests in multiple subdirectories should be contained in tests/conftest.py. You can have multiple conftest.py files; for example, you can have one at tests and one for each subdirectory under tests.

If you haven't already done so, you can download a copy of the source code for this project on the book's website.[5] Alternatively, you can work on your own project with a similar structure.

Installing a Package Locally

The test file, tests/test_task.py, contains the tests we worked on in *Running pytest*, in files test_three.py and test_four.py. I've just renamed it here to something that makes more sense for what it's testing and copied everything into one file. I also removed the definition of the Task data structure, because that really belongs in api.py.

Here is test_task.py:

ch2/tasks_proj/tests/unit/test_task.py

```
    """Test the Task data type."""
    from tasks import Task


    def test_asdict():
    """_asdict() should return a dictionary."""
    t_task = Task('do something', 'okken', True, 21)
    t_dict = t_task._asdict()
    expected = {'summary': 'do something',
    'owner': 'okken',
    'done': True,
    'id': 21}
    assert t_dict == expected


    def test_replace():
    """replace() should change passed in fields."""
    t_before = Task('finish book', 'brian', False)
    t_after = t_before._replace(id=10, done=True)
    t_expected = Task('finish book', 'brian', True, 10)
    assert t_after == t_expected


    def test_defaults():
    """Using no parameters should invoke defaults."""
    t1 = Task()
    t2 = Task(None, None, False, None)
    assert t1 == t2


    def test_member_access():
    """Check .field functionality of namedtuple."""
    t = Task('buy milk', 'brian')
    assert t.summary == 'buy milk'
    assert t.owner == 'brian'
    assert (t.done, t.id) == (False, None)
```

The test_task.py file has this import statement:

```
    from tasks import Task
```

The best way to allow the tests to be able to import tasks or from tasks import something is to install tasks locally using pip. This is possible because there’s a setup.py file present to direct

pip.

Install tasks either by running pip install . or pip install -e . from the tasks_proj directory. Or you can run pip install -e tasks_proj from one directory up:

```
$ cd /path/to/code
$ pip install ./tasks_proj/
$ pip install --no-cache-dir ./tasks_proj/
Processing ./tasks_proj
Collecting click (from tasks==0.1.0)
Downloading click-6.7-py2.py3-none-any.whl (71kB)
 ...
Collecting tinydb (from tasks==0.1.0)
Downloading tinydb-3.4.0.tar.gz
Collecting six (from tasks==0.1.0)
Downloading six-1.10.0-py2.py3-none-any.whl
Installing collected packages: click, tinydb, six, tasks
Running setup.py install for tinydb ... done
Running setup.py install for tasks ... done
Successfully installed click-6.7 six-1.10.0 tasks-0.1.0 tinydb-3.4.0
```

If you only want to run tests against tasks, this command is fine. If you want to be able to modify the source code while tasks is installed, you need to install it with the -e option (for “editable”):

```
$ pip install -e ./tasks_proj/
Obtaining file:///path/to/code/tasks_proj
Requirement already satisfied: click in
/path/to/venv/lib/python3.6/site-packages (from tasks==0.1.0)
Requirement already satisfied: tinydb in
/path/to/venv/lib/python3.6/site-packages (from tasks==0.1.0)
Requirement already satisfied: six in
/path/to/venv/lib/python3.6/site-packages (from tasks==0.1.0)
Installing collected packages: tasks
Found existing installation: tasks 0.1.0
Uninstalling tasks-0.1.0:
Successfully uninstalled tasks-0.1.0
Running setup.py develop for tasks
Successfully installed tasks
```

Now let’s try running tests:

```
$ cd /path/to/code/ch2/tasks_proj/tests/unit
$ pytest test_task.py
==================== test session starts ====================
```

```
collected 4 items

test_task.py ....

==================== 4 passed in 0.01 seconds ====================
```

The import worked! The rest of our tests can now safely use import tasks. Now let’s write some tests.

Using assert Statements

When you write test functions, the normal Python assert statement is your primary tool to communicate test failure. The simplicity of this within pytest is brilliant. It's what drives a lot of developers to use pytest over other frameworks.

If you've used any other testing framework, you've probably seen various assert helper functions. For example, the following is a list of a few of the assert forms and assert helper functions:

pytest	unittest
assert something	assertTrue(something)
assert a == b	assertEqual(a, b)
assert a <= b	assertLessEqual(a, b)
…	…

With pytest, you can use assert <expression> with any expression. If the expression would evaluate to False if converted to a bool, the test would fail.

pytest includes a feature called assert rewriting that intercepts assert calls and replaces them with something that can tell you more about why your assertions failed. Let's see how helpful this rewriting is by looking at a few assertion failures:

ch2/tasks_proj/tests/unit/test_task_fail.py

```
"""Use the Task type to show test failures."""
from tasks import Task


def test_task_equality():
"""Different tasks should not be equal."""
t1 = Task('sit there', 'brian')
t2 = Task('do something', 'okken')
assert t1 == t2


def test_dict_equality():
```

```
    """Different tasks compared as dicts should not be equal."""
    t1_dict = Task('make sandwich', 'okken')._asdict()
    t2_dict = Task('make sandwich', 'okkem')._asdict()
    assert t1_dict == t2_dict
```

All of these tests fail, but what's interesting is the traceback information:

```
$ cd /path/to/code/ch2/tasks_proj/tests/unit
$ pytest test_task_fail.py
==================== test session starts ====================
collected 2 items

test_task_fail.py FF

========================== FAILURES ==========================
_____________________ test_task_equality _____________________

def test_task_equality():
t1 = Task('sit there', 'brian')
t2 = Task('do something', 'okken')
> assert t1 == t2
E AssertionError: assert Task(summary=...alse, id=None) ==
Task(summary='...alse, id=None)
E At index 0 diff: 'sit there' != 'do something'
E Use -v to get the full diff

test_task_fail.py:6: AssertionError
_____________________ test_dict_equality _____________________

def test_dict_equality():
t1_dict = Task('make sandwich', 'okken')._asdict()
t2_dict = Task('make sandwich', 'okkem')._asdict()
> assert t1_dict == t2_dict
E AssertionError: assert OrderedDict([...('id', None)]) ==
OrderedDict([(...('id', None)])
E Omitting 3 identical items, use -v to show
E Differing items:
E {'owner': 'okken'} != {'owner': 'okkem'}
E Use -v to get the full diff

test_task_fail.py:11: AssertionError
================== 2 failed in 0.06 seconds ==================
```

Wow. That's a lot of information. For each failing test, the exact line of failure is shown with a > pointing to the failure. The E lines show you extra information about the assert failure to help you figure out what went wrong.

I intentionally put two mismatches in test_task_equality(), but only the first was shown in the previous code. Let's try it again with the -v flag, as suggested in the error message:

```
$ pytest -v test_task_fail.py::test_task_equality
==================== test session starts ====================
collected 3 items

test_task_fail.py::test_task_equality FAILED

============================ FAILURES ============================
_____________________ test_task_equality _____________________

def test_task_equality():
t1 = Task('sit there', 'brian')
t2 = Task('do something', 'okken')
> assert t1 == t2
E AssertionError: assert Task(summary=...alse, id=None) ==
Task(summary='...alse, id=None)
E At index 0 diff: 'sit there' != 'do something'
E Full diff:
E - Task(summary='sit there', owner='brian', done=False, id=None)
E ? ^^^ ^^^ ^^^^
E + Task(summary='do something', owner='okken', done=False, id=None)
E ? +++ ^^^ ^^^ ^^^^

test_task_fail.py:6: AssertionError
================== 1 failed in 0.07 seconds ==================
```

Well, I think that's pretty darned cool. pytest not only found both differences, but it also showed us exactly where the differences are.

This example only used equality assert; many more varieties of assert statements with awesome trace debug information are found on the pytest.org website.[6]

Expecting Exceptions

Exceptions may be raised in a few places in the Tasks API. Let's take a quick peek at the functions found in tasks/api.py:

```
  def add(task): # type: (Task) -> int
  def get(task_id): # type: (int) -> Task
  def list_tasks(owner=None): # type: (str|None) -> list of Task
  def count(): # type: (None) -> int
  def update(task_id, task): # type: (int, Task) -> None
  def delete(task_id): # type: (int) -> None
  def delete_all(): # type: () -> None
  def unique_id(): # type: () -> int
  def start_tasks_db(db_path, db_type): # type: (str, str) -> None
  def stop_tasks_db(): # type: () -> None
```

There's an agreement between the CLI code in cli.py and the API code in api.py as to what types will be sent to the API functions. These API calls are a place where I'd expect exceptions to be raised if the type is wrong.

To make sure these functions raise exceptions if called incorrectly, let's use the wrong type in a test function to intentionally cause TypeError exceptions, and use with pytest.raises(<expected exception>), like this:

ch2/tasks_proj/tests/func/test_api_exceptions.py

```
  import pytest
  import tasks


  def test_add_raises():
  """add() should raise an exception with wrong type param."""
  with pytest.raises(TypeError):
  tasks.add(task='not a Task object')
```

In test_add_raises(), the with pytest.raises(TypeError): statement says that whatever is in the next block of code should raise a TypeError exception. If no exception is raised, the test fails. If the test raises a different exception, it fails.

We just checked for the type of exception in test_add_raises(). You can also check the parameters to the exception. For start_tasks_db(db_path, db_type), not only does db_type need to be a string, it really has to be either 'tiny' or 'mongo'. You can check to make sure the exception message is correct by adding as excinfo:

ch2/tasks_proj/tests/func/test_api_exceptions.py

```
  def test_start_tasks_db_raises():
```

```
    """Make sure unsupported db raises an exception."""
    with pytest.raises(ValueError) as excinfo:
    tasks.start_tasks_db('some/great/path', 'mysql')
    exception_msg = excinfo.value.args[0]
    assert exception_msg == "db_type must be a 'tiny' or 'mongo'"
```

This allows us to look at the exception more closely. The variable name you put after as (excinfo in this case) is filled with information about the exception, and is of type ExceptionInfo.

In our case, we want to make sure the first (and only) parameter to the exception matches a string.

Marking Test Functions

pytest provides a cool mechanism to let you put markers on test functions. A test can have more than one marker, and a marker can be on multiple tests.

Markers make sense after you see them in action. Let's say we want to run a subset of our tests as a quick "smoke test" to get a sense for whether or not there is some major break in the system. Smoke tests are by convention not all-inclusive, thorough test suites, but a select subset that can be run quickly and give a developer a decent idea of the health of all parts of the system.

To add a smoke test suite to the Tasks project, we can add @mark.pytest.smoke to some of the tests. Let's add it to a couple of tests in test_api_exceptions.py (note that the markers smoke and get aren't built into pytest; I just made them up):

ch2/tasks_proj/tests/func/test_api_exceptions.py

```
@pytest.mark.smoke
def test_list_raises():
"""list() should raise an exception with wrong type param."""
with pytest.raises(TypeError):
tasks.list_tasks(owner=123)


@pytest.mark.get
@pytest.mark.smoke
def test_get_raises():
"""get() should raise an exception with wrong type param."""
with pytest.raises(TypeError):
tasks.get(task_id='123')
```

Now, let's run just those tests that are marked with -m marker_name:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v -m 'smoke' test_api_exceptions.py
==================== test session starts ====================
collected 7 items

test_api_exceptions.py::test_list_raises PASSED
test_api_exceptions.py::test_get_raises PASSED

====================== 5 tests deselected ======================
============ 2 passed, 5 deselected in 0.03 seconds ============
$ pytest -v -m 'get' test_api_exceptions.py
==================== test session starts ====================
```

```
collected 7 items

test_api_exceptions.py::test_get_raises PASSED

====================== 6 tests deselected ======================
============ 1 passed, 6 deselected in 0.01 seconds ============
```

Remember that -v is short for --verbose and lets us see the names of the tests that are run. Using -m 'smoke' runs both tests marked with @pytest.mark.smoke. Using -m 'get' runs the one test marked with @pytest.mark.get. Pretty straightforward.

It gets better. The expression after -m can use and, or, and not to combine multiple markers:

```
$ pytest -v -m 'smoke and get' test_api_exceptions.py
==================== test session starts ====================
collected 7 items

test_api_exceptions.py::test_get_raises PASSED

====================== 6 tests deselected ======================
============ 1 passed, 6 deselected in 0.03 seconds ============
```

That time we only ran the test that had both smoke and get markers. We can use not as well:

```
$ pytest -v -m 'smoke and not get' test_api_exceptions.py
==================== test session starts ====================
collected 7 items

test_api_exceptions.py::test_list_raises PASSED

====================== 6 tests deselected ======================
============ 1 passed, 6 deselected in 0.03 seconds ============
```

The addition of -m 'smoke and not get' selected the test that was marked with @pytest.mark.smoke but not @pytest.mark.get.

Filling Out the Smoke Test

The previous tests don't seem like a reasonable smoke test suite yet. We haven't actually touched the database or added any tasks. Surely a smoke test would do that.

Let's add a couple of tests that look at adding a task, and use one of them as part of our smoke test suite:

ch2/tasks_proj/tests/func/test_add.py

```
import pytest
```

```
import tasks
from tasks import Task


def test_add_returns_valid_id():
"""tasks.add(<valid task>) should return an integer."""
# GIVEN an initialized tasks db
# WHEN a new task is added
# THEN returned task_id is of type int
new_task = Task('do something')
task_id = tasks.add(new_task)
assert isinstance(task_id, int)


@pytest.mark.smoke
def test_added_task_has_id_set():
"""Make sure the task_id field is set by tasks.add()."""
# GIVEN an initialized tasks db
# AND a new task is added
new_task = Task('sit in chair', owner='me', done=True)
task_id = tasks.add(new_task)

# WHEN task is retrieved
task_from_db = tasks.get(task_id)

# THEN task_id matches id field
assert task_from_db.id == task_id
```

Both of these tests have the comment GIVEN an initialized tasks db, and yet there is no database initialized in the test. We can define a fixture to get the database initialized before the test and cleaned up after the test:

ch2/tasks_proj/tests/func/test_add.py

```
@pytest.fixture(autouse=True)
def initialized_tasks_db(tmpdir):
"""Connect to db before testing, disconnect after."""
# Setup : start db
tasks.start_tasks_db(str(tmpdir), 'tiny')

yield # this is where the testing happens

# Teardown : stop db
tasks.stop_tasks_db()
```

The fixture, tmpdir, used in this example is a builtin fixture. You'll learn all about builtin fixtures in Chapter 4, *Builtin Fixtures*, and you'll learn about writing your own fixtures and how they work in Chapter 3, *pytest Fixtures*, including the autouse parameter used here.

autouse as used in our test indicates that all tests in this file will use the fixture. The code before the yield runs before each test; the code after the yield runs after the test. The yield can return data to the test if desired. You'll look at all that and more in later chapters, but here we need some way to set up the database for testing, so I couldn't wait any longer to show you a fixture. (pytest also supports old-fashioned setup and teardown functions, like what is used in unittest and nose, but they are not nearly as fun. However, if you are curious, they are described in Appendix 5, *xUnit Fixtures*.)

Let's set aside fixture discussion for now and go to the top of the project and run our smoke test suite:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest -v -m 'smoke'
====================== test session starts ======================
collected 56 items

tests/func/test_add.py::test_added_task_has_id_set PASSED
tests/func/test_api_exceptions.py::test_list_raises PASSED
tests/func/test_api_exceptions.py::test_get_raises PASSED

====================== 53 tests deselected ======================
=========== 3 passed, 53 deselected in 0.11 seconds ============
```

This shows that marked tests from different files can all run together.

Skipping Tests

While the markers discussed in *Marking Test Functions* were names of your own choosing, pytest includes a few helpful builtin markers: skip, skipif, and xfail. I'll discuss skip and skipif in this section, and xfail in the next.

The skip and skipif markers enable you to skip tests you don't want to run. For example, let's say we weren't sure how tasks.unique_id() was supposed to work. Does each call to it return a different number? Or is it just a number that doesn't exist in the database already?

First, let's write a test (note that the initialized_tasks_db fixture is in this file, too; it's just not shown here):

ch2/tasks_proj/tests/func/test_unique_id_1.py

```
import pytest
import tasks


def test_unique_id():
"""Calling unique_id() twice should return different numbers."""
id_1 = tasks.unique_id()
id_2 = tasks.unique_id()
assert id_1 != id_2
```

Then give it a run:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest test_unique_id_1.py
===================== test session starts ======================
collected 1 item s

test_unique_id_1.py F

============================ FAILURES ============================
_____________________________ test_unique_id _____________________________

def test_unique_id():
"""Calling unique_id() twice should return different numbers."""
id_1 = tasks.unique_id()
id_2 = tasks.unique_id()
> assert id_1 != id_2
E assert 1 != 1
```

```
test_unique_id_1.py:12: AssertionError
================== 1 failed in 0.06 seconds ==================
```

Hmm. Maybe we got that wrong. After looking at the API a bit more, we see that the docstring says """Return an integer that does not exist in the db.""".

We could just change the test. But instead, let's just mark the first one to get skipped for now:

ch2/tasks_proj/tests/func/test_unique_id_2.py

```
@pytest.mark.skip(reason='misunderstood the API')
def test_unique_id_1():
    """Calling unique_id() twice should return different numbers."""
    id_1 = tasks.unique_id()
    id_2 = tasks.unique_id()
    assert id_1 != id_2


def test_unique_id_2():
    """unique_id() should return an unused id."""
    ids = []
    ids.append(tasks.add(Task('one')))
    ids.append(tasks.add(Task('two')))
    ids.append(tasks.add(Task('three')))
    # grab a unique id
    uid = tasks.unique_id()
    # make sure it isn't in the list of existing ids
    assert uid not in ids
```

Marking a test to be skipped is as simple as adding @pytest.mark.skip() just above the test function.

Let's run again:

```
$ pytest -v test_unique_id_2.py
========================== test session starts ==========================
collected 2 items

test_unique_id_2.py::test_unique_id_1 SKIPPED
test_unique_id_2.py::test_unique_id_2 PASSED

================== 1 passed, 1 skipped in 0.02 seconds ==================
```

Now, let's say that for some reason we decide the first test should be valid also, and we intend to make that work in version 0.2.0 of the package. We can leave the test in place and use skipif instead:

ch2/tasks_proj/tests/func/test_unique_id_3.py

```
@pytest.mark.skipif(tasks.__version__ < '0.2.0',
reason='not supported until version 0.2.0')
def test_unique_id_1():
"""Calling unique_id() twice should return different numbers."""
id_1 = tasks.unique_id()
id_2 = tasks.unique_id()
assert id_1 != id_2
```

The expression we pass into skipif() can be any valid Python expression. In this case, we're checking the package version.

We included reasons in both skip and skipif. It's not required in skip, but it is required in skipif. I like to include a reason for every skip, skipif, or xfail.

Here's the output of the changed code:

```
$ pytest test_unique_id_3.py
========================== test session starts ==========================
collected 2 items

test_unique_id_3.py s.

=================== 1 passed, 1 skipped in 0.02 seconds ===================
```

The s. shows that one test was skipped and one test passed.

We can see which one with -v:

```
$ pytest -v test_unique_id_3.py
========================== test session starts ==========================
collected 2 items

test_unique_id_3.py::test_unique_id_1 SKIPPED
test_unique_id_3.py::test_unique_id_2 PASSED

=================== 1 passed, 1 skipped in 0.03 seconds ===================
```

But we still don't know why. We can see those reasons with -rs:

```
$ pytest -rs test_unique_id_3.py
======================== test session starts ========================
collected 2 items

test_unique_id_3.py s.
```

```
========================= short test summary info ==========================
SKIP [1] func/test_unique_id_3.py:5: not supported until version 0.2.0

================== 1 passed, 1 skipped in 0.03 seconds ==================
```

The -r chars option has this help text:

```
$ pytest --help
...
-r chars

show extra test summary info as specified by chars
(f)ailed, (E)error, (s)skipped, (x)failed, (X)passed,
(p)passed, (P)passed with output, (a)all except pP.
...
```

It’s not only helpful for understanding test skips, but also you can use it for other test outcomes as well.

Marking Tests as Expecting to Fail

With the skip and skipif markers, a test isn't even attempted if skipped. With the xfail marker, we are telling pytest to run a test function, but that we expect it to fail. Let's modify our unique_id() test again to use xfail:

ch2/tasks_proj/tests/func/test_unique_id_4.py

```
@pytest.mark.xfail(tasks.__version__ < '0.2.0',
reason='not supported until version 0.2.0')
def test_unique_id_1():
"""Calling unique_id() twice should return different numbers."""
id_1 = tasks.unique_id()
id_2 = tasks.unique_id()
assert id_1 != id_2


@pytest.mark.xfail()
def test_unique_id_is_a_duck():
"""Demonstrate xfail."""
uid = tasks.unique_id()
assert uid == 'a duck'


@pytest.mark.xfail()
def test_unique_id_not_a_duck():
"""Demonstrate xpass."""
uid = tasks.unique_id()
assert uid != 'a duck'
```

The first test is the same as before, but with xfail. The next two tests are listed as xfail, and differ only by == vs. !=. So one of them is bound to pass.

Running this shows:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest test_unique_id_4.py
======================== test session starts ========================
collected 4 items

test_unique_id_4.py xxX.

========== 1 passed, 2 xfailed, 1 xpassed in 0.07 seconds ===========
```

The x is for XFAIL, which means “expected to fail.” The capital X is for XPASS or “expected to fail but passed.”

--verbose lists longer descriptions:

```
$ pytest -v test_unique_id_4.py
======================== test session starts ========================
collected 4 items

test_unique_id_4.py::test_unique_id_1 xfail
test_unique_id_4.py::test_unique_id_is_a_duck xfail
test_unique_id_4.py::test_unique_id_not_a_duck XPASS
test_unique_id_4.py::test_unique_id_2 PASSED

========== 1 passed, 2 xfailed, 1 xpassed in 0.08 seconds ===========
```

You can configure pytest to report the tests that pass but were marked with xfail to be reported as FAIL. This is done in a pytest.ini file:

```
[pytest]
xfail_strict=true
```

I’ll discuss pytest.ini more in Chapter 6, *Configuration*.

Running a Subset of Tests

I've talked about how you can place markers on tests and run tests based on markers. You can run a subset of tests in several other ways. You can run all of the tests, or you can select a single directory, file, class within a file, or an individual test in a file or class. You haven't seen test classes used yet, so you'll look at one in this section. You can also use an expression to match test names. Let's take a look at these.

A Single Directory

To run all the tests from one directory, use the directory as a parameter to pytest:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest tests/func --tb=no
====================== test session starts ======================
collected 50 items

tests/func/test_add.py ..
tests/func/test_add_variety.py .............................
tests/func/test_api_exceptions.py .......
tests/func/test_unique_id_1.py F
tests/func/test_unique_id_2.py s.
tests/func/test_unique_id_3.py s.
tests/func/test_unique_id_4.py xxX.

1 failed, 44 passed, 2 skipped, 2 xfailed, 1 xpassed in 0.26 seconds
```

An important trick to learn is that using -v gives you the syntax for how to run a specific directory, class, and test.

```
$ pytest -v tests/func --tb=no
====================== test session starts ======================
collected 50 items

tests/func/test_add.py::test_add_returns_valid_id PASSED
tests/func/test_add.py::test_added_task_has_id_set PASSED
...
tests/func/test_api_exceptions.py::test_add_raises PASSED
tests/func/test_api_exceptions.py::test_list_raises PASSED
tests/func/test_api_exceptions.py::test_get_raises PASSED
...
tests/func/test_unique_id_1.py::test_unique_id FAILED
```

```
tests/func/test_unique_id_2.py::test_unique_id_1 SKIPPED
tests/func/test_unique_id_2.py::test_unique_id_2 PASSED
...
tests/func/test_unique_id_4.py::test_unique_id_1 xfail
tests/func/test_unique_id_4.py::test_unique_id_is_a_duck xfail
tests/func/test_unique_id_4.py::test_unique_id_not_a_duck XPASS
tests/func/test_unique_id_4.py::test_unique_id_2 PASSED

1 failed, 44 passed, 2 skipped, 2 xfailed, 1 xpassed in 0.30 seconds
```

You'll see the syntax listed here in the next few examples.

A Single Test File/Module

To run a file full of tests, list the file with the relative path as a parameter to pytest:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest tests/func/test_add.py
============================ test session starts ============================
collected 2 items

tests/func/test_add.py ..

========================= 2 passed in 0.05 seconds =========================
```

We've been doing this for a while.

A Single Test Function

To run a single test function, add :: and the test function name:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest -v tests/func/test_add.py::test_add_returns_valid_id
============================ test session starts ============================
collected 3 items

tests/func/test_add.py::test_add_returns_valid_id PASSED

========================= 1 passed in 0.02 seconds =========================
```

Use -v so you can see which function was run.

A Single Test Class

Test classes are a way to group tests that make sense to be grouped together. Here's an example:

ch2/tasks_proj/tests/func/test_api_exceptions.py

```
class TestUpdate():
"""Test expected exceptions with tasks.update()."""

def test_bad_id(self):
"""A non-int id should raise an excption."""
with pytest.raises(TypeError):
tasks.update(task_id={'dict instead': 1},
task=tasks.Task())

def test_bad_task(self):
"""A non-Task task should raise an excption."""
with pytest.raises(TypeError):
tasks.update(task_id=1, task='not a task')
```

Since these are two related tests that both test the update() function, it's reasonable to group them in a class. To run just this class, do like we did with functions and add ::, then the class name to the file parameter:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest -v tests/func/test_api_exceptions.py::TestUpdate
========================== test session starts ==========================
collected 7 items

tests/func/test_api_exceptions.py::TestUpdate::test_bad_id PASSED
tests/func/test_api_exceptions.py::TestUpdate::test_bad_task PASSED

======================== 2 passed in 0.03 seconds ========================
```

A Single Test Method of a Test Class

If you don't want to run all of a test class—just one method—just add another :: and the method name:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest -v tests/func/test_api_exceptions.py::TestUpdate::test_bad_id
====================== test session starts ======================
collected 1 item

tests/func/test_api_exceptions.py::TestUpdate::test_bad_id PASSED

================== 1 passed in 0.03 seconds ==================
```

Grouping Syntax Shown by Verbose Listing



Remember that the syntax for how to run a subset of tests by directory, file, function, class, and method doesn't have to be memorized. The format is the same as the test function listing when you run pytest -v.

A Set of Tests Based on Test Name

The -k option enables you to pass in an expression to run tests that have certain names specified by the expression as a substring of the test name. You can use and, or, and not in your expression to create complex expressions.

For example, we can run all of the functions that have _raises in their name:

```
$ cd /path/to/code/ch2/tasks_proj
$ pytest -v -k _raises
==================== test session starts ====================
collected 56 items

tests/func/test_api_exceptions.py::test_add_raises PASSED
tests/func/test_api_exceptions.py::test_list_raises PASSED
tests/func/test_api_exceptions.py::test_get_raises PASSED
tests/func/test_api_exceptions.py::test_delete_raises PASSED
tests/func/test_api_exceptions.py::test_start_tasks_db_raises PASSED

==================== 51 tests deselected ====================
========== 5 passed, 51 deselected in 0.07 seconds ===========
```

We can use and and not to get rid of the test_delete_raises() from the session:

```
$ pytest -v -k "_raises and not delete"
==================== test session starts ====================
collected 56 items

tests/func/test_api_exceptions.py::test_add_raises PASSED
tests/func/test_api_exceptions.py::test_list_raises PASSED
tests/func/test_api_exceptions.py::test_get_raises PASSED
tests/func/test_api_exceptions.py::test_start_tasks_db_raises PASSED

==================== 52 tests deselected ====================
========== 4 passed, 52 deselected in 0.06 seconds ===========
```

In this section, you learned how to run specific test files, directories, classes, and functions, and how to use expressions with -k to run specific sets of tests. In the next section, you'll learn how one test function can turn into many test cases by allowing the test to run multiple times with different test data.

Parametrized Testing

Sending some values through a function and checking the output to make sure it's correct is a common pattern in software testing. However, calling a function once with one set of values and one check for correctness isn't enough to fully test most functions. Parametrized testing is a way to send multiple sets of data through the same test and have pytest report if any of the sets failed.

To help understand the problem parametrized testing is trying to solve, let's take a simple test for add():

ch2/tasks_proj/tests/func/test_add_variety.py

```
import pytest
import tasks
from tasks import Task


def test_add_1():
"""tasks.get() using id returned from add() works."""
task = Task('breathe', 'BRIAN', True)
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
# everything but the id should be the same
assert equivalent(t_from_db, task)


def equivalent(t1, t2):
"""Check two tasks for equivalence."""
# Compare everything but the id field
return ((t1.summary == t2.summary) and
(t1.owner == t2.owner) and
(t1.done == t2.done))


@pytest.fixture(autouse=True)
def initialized_tasks_db(tmpdir):
"""Connect to db before testing, disconnect after."""
tasks.start_tasks_db(str(tmpdir), 'tiny')
yield
tasks.stop_tasks_db()
```

When a Task object is created, its id field is set to None. After it's added and retrieved from the database, the id field will be set. Therefore, we can't just use == to check to see if our task was added and retrieved correctly. The equivalent() helper function checks all but the id field. The

autouse fixture is included to make sure the database is accessible. Let's make sure the test passes:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_1
==================== test session starts ====================
collected 1 item

test_add_variety.py::test_add_1 PASSED

=================== 1 passed in 0.03 seconds ===================
```

The test seems reasonable. However, it's just testing one example task. What if we want to test lots of variations of a task? No problem. We can use @pytest.mark.parametrize(argnames, argvalues) to pass lots of data through the same test, like this:

ch2/tasks_proj/tests/func/test_add_variety.py

```
@pytest.mark.parametrize('task',
[Task('sleep', done=True),
Task('wake', 'brian'),
Task('breathe', 'BRIAN', True),
Task('exercise', 'BrIaN', False)])
def test_add_2(task):
"""Demonstrate parametrize with one parameter."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
```

The first argument to parametrize() is a string with a comma-separated list of names—'task', in our case. The second argument is a list of values, which in our case is a list of Task objects. pytest will run this test once for each task and report each as a separate test:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_2
==================== test session starts ====================
collected 4 items

test_add_variety.py::test_add_2[task0] PASSED
test_add_variety.py::test_add_2[task1] PASSED
test_add_variety.py::test_add_2[task2] PASSED
test_add_variety.py::test_add_2[task3] PASSED

=================== 4 passed in 0.05 seconds ===================
```

This use of parametrize() works for our purposes. However, let's pass in the tasks as tuples to see

how multiple test parameters would work:

ch2/tasks_proj/tests/func/test_add_variety.py

```
@pytest.mark.parametrize('summary, owner, done',
[('sleep', None, False),
('wake', 'brian', False),
('breathe', 'BRIAN', True),
('eat eggs', 'BrIaN', False),
])
def test_add_3(summary, owner, done):
"""Demonstrate parametrize with multiple parameters."""
task = Task(summary, owner, done)
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
```

When you use types that are easy for pytest to convert into strings, the test identifier uses the parameter values in the report to make it readable:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_3
==================== test session starts ====================
collected 4 items

test_add_variety.py::test_add_3[sleep-None-False] PASSED
test_add_variety.py::test_add_3[wake-brian-False] PASSED
test_add_variety.py::test_add_3[breathe-BRIAN-True] PASSED
test_add_variety.py::test_add_3[eat eggs-BrIaN-False] PASSED

=================== 4 passed in 0.05 seconds ===================
```

You can use that whole test identifier—called a node in pytest terminology—to re-run the test if you want:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_3[sleep-None-False]
==================== test session starts ====================
collected 1 item

test_add_variety.py::test_add_3[sleep-None-False] PASSED

=================== 1 passed in 0.02 seconds ===================
```

Be sure to use quotes if there are spaces in the identifier:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v "test_add_variety.py::test_add_3[eat eggs-BrIaN-False]"
==================== test session starts ====================
collected 1 item

test_add_variety.py::test_add_3[eat eggs-BrIaN-False] PASSED

=================== 1 passed in 0.03 seconds ===================
```

Now let's go back to the list of tasks version, but move the task list to a variable outside the function:

ch2/tasks_proj/tests/func/test_add_variety.py

```
tasks_to_try = (Task('sleep', done=True),
Task('wake', 'brian'),
Task('wake', 'brian'),
Task('breathe', 'BRIAN', True),
Task('exercise', 'BrIaN', False))


@pytest.mark.parametrize('task', tasks_to_try)
def test_add_4(task):
"""Slightly different take."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
```

It's convenient and the code looks nice. But the readability of the output is hard to interpret:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_4
==================== test session starts ====================
collected 5 items

test_add_variety.py::test_add_4[task0] PASSED
test_add_variety.py::test_add_4[task1] PASSED
test_add_variety.py::test_add_4[task2] PASSED
test_add_variety.py::test_add_4[task3] PASSED
test_add_variety.py::test_add_4[task4] PASSED

=================== 5 passed in 0.05 seconds ===================
```

The readability of the multiple parameter version is nice, but so is the list of Task objects. To compromise, we can use the ids optional parameter to parametrize() to make our own identifiers

for each task data set. The ids parameter needs to be a list of strings the same length as the number of data sets. However, because we assigned our data set to a variable name, tasks_to_try, we can use it to generate ids:

ch2/tasks_proj/tests/func/test_add_variety.py

```
task_ids = ['Task({},{},{})'.format(t.summary, t.owner, t.done)
for t in tasks_to_try]


@pytest.mark.parametrize('task', tasks_to_try, ids=task_ids)
def test_add_5(task):
"""Demonstrate ids."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
```

Let's run that and see how it looks:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_5
==================== test session starts ====================
collected 5 items

test_add_variety.py::test_add_5[Task(sleep,None,True)] PASSED
test_add_variety.py::test_add_5[Task(wake,brian,False)0] PASSED
test_add_variety.py::test_add_5[Task(wake,brian,False)1] PASSED
test_add_variety.py::test_add_5[Task(breathe,BRIAN,True)] PASSED
test_add_variety.py::test_add_5[Task(exercise,BrIaN,False)] PASSED

================== 5 passed in 0.04 seconds ==================
```

And these test identifiers can be used to run tests:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v "test_add_variety.py::test_add_5[Task(exercise,BrIaN,False)]"
==================== test session starts ====================
collected 1 item

test_add_variety.py::test_add_5[Task(exercise,BrIaN,False)] PASSED

================== 1 passed in 0.03 seconds ==================
```

We definitely need quotes for these identifiers; otherwise, the brackets and parentheses will confuse the shell.

You can apply parametrize() to classes as well. When you do that, the same data sets will be sent to all test methods in the class:

ch2/tasks_proj/tests/func/test_add_variety.py

```
@pytest.mark.parametrize('task', tasks_to_try, ids=task_ids)
class TestAdd():
"""Demonstrate parametrize and test classes."""

def test_equivalent(self, task):
"""Similar test, just within a class."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
def test_valid_id(self, task):
"""We can use the same data or multiple tests."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert t_from_db.id == task_id
```

Here it is in action:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::TestAdd
==================== test session starts ====================
collected 10 items

test_add_variety.py::TestAdd::test_equivalent[Task(sleep,None,True)] PASSED
test_add_variety.py::TestAdd::test_equivalent[Task(wake,brian,False)0] PASSED
test_add_variety.py::TestAdd::test_equivalent[Task(wake,brian,False)1] PASSED
test_add_variety.py::TestAdd::test_equivalent[Task(breathe,BRIAN,True)] PASSED
test_add_variety.py::TestAdd::test_equivalent[Task(exercise,BrIaN,False)] PASSED
test_add_variety.py::TestAdd::test_valid_id[Task(sleep,None,True)] PASSED
test_add_variety.py::TestAdd::test_valid_id[Task(wake,brian,False)0] PASSED
test_add_variety.py::TestAdd::test_valid_id[Task(wake,brian,False)1] PASSED
test_add_variety.py::TestAdd::test_valid_id[Task(breathe,BRIAN,True)] PASSED
test_add_variety.py::TestAdd::test_valid_id[Task(exercise,BrIaN,False)] PASSED

================== 10 passed in 0.08 seconds ==================
```

You can also identify parameters by including an id right alongside the parameter value when passing in a list within the @pytest.mark.parametrize() decorator. You do this with pytest.param(<value>, id="something") syntax:

ch2/tasks_proj/tests/func/test_add_variety.py

```
@pytest.mark.parametrize('task', [
pytest.param(Task('create'), id='just summary'),
pytest.param(Task('inspire', 'Michelle'), id='summary/owner'),
pytest.param(Task('encourage', 'Michelle', True), id='summary/owner/done')])
def test_add_6(task):
"""Demonstrate pytest.param and id."""
task_id = tasks.add(task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, task)
```

In action:

```
$ cd /path/to/code/ch2/tasks_proj/tests/func
$ pytest -v test_add_variety.py::test_add_6
================== test session starts ==================
collected 3 items

test_add_variety.py::test_add_6[just summary] PASSED
test_add_variety.py::test_add_6[summary/owner] PASSED
test_add_variety.py::test_add_6[summary/owner/done] PASSED

================= 3 passed in 0.05 seconds =================
```

This is useful when the id cannot be derived from the parameter value.

Exercises

1. Download the project for this chapter, tasks_proj, from the book's webpage[7] and make sure you can install it locally with pip install /path/to/tasks_proj.
2. Explore the tests directory.
3. Run pytest with a single file.
4. Run pytest against a single directory, such as tasks_proj/tests/func. Use pytest to run tests individually as well as a directory full at a time. There are some failing tests there. Do you understand why they fail?
5. Add xfail or skip markers to the failing tests until you can run pytest from the tests directory with no arguments and no failures.
6. We don't have any tests for tasks.count() yet, among other functions. Pick an untested API function and think of which test cases we need to have to make sure it works correctly.
7. What happens if you try to add a task with the id already set? There are some missing exception tests in test_api_exceptions.py. See if you can fill in the missing exceptions. (It's okay to look at api.py for this exercise.)

What’s Next

You’ve run through a lot of the power of pytest in this chapter. Even with just what’s covered here, you can start supercharging your test suites. In many of the examples, you used a fixture called initialized_tasks_db. Fixtures can separate retrieving and/or generating test data from the real guts of a test function. They can also separate common code so that multiple test functions can use the same setup. In the next chapter, you’ll take a deep dive into the wonderful world of pytest fixtures.

Footnotes

[5]

https://pragprog.com/titles/bopytest/source_code

[6]

http://doc.pytest.org/en/latest/example/reportingdemo.html

[7]

https://pragprog.com/titles/bopytest/source_code

Copyright © 2017, The Pragmatic Bookshelf.